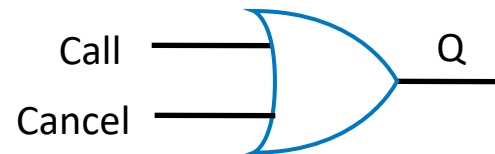


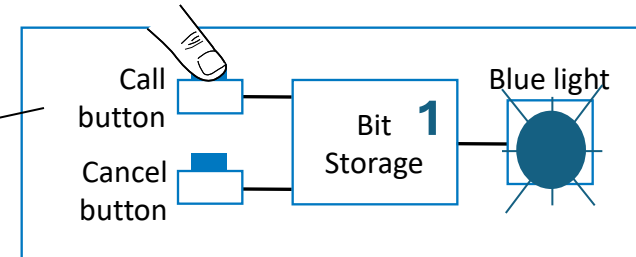
# Storing One Bit

- Flight attendant call button
  - Press call: light turns on
    - **Stays on** after button released
  - Press cancel: light turns off
    - Stays off after button released
- Logic gate circuit to implement this?

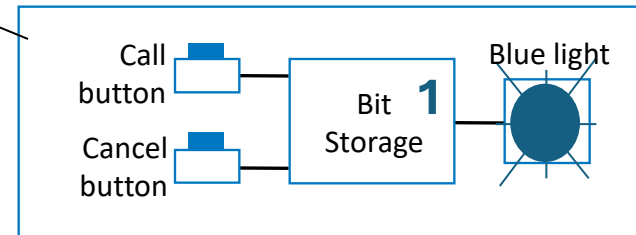


Doesn't work.  $Q=1$  when  $\text{Call}=1$ , but doesn't stay 1 when Call returns to 0

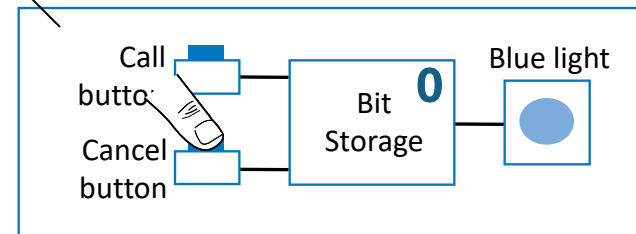
*Need some form of "feedback" in the circuit*



1. Call button pressed – light turns on



2. Call button released – light **stays on**

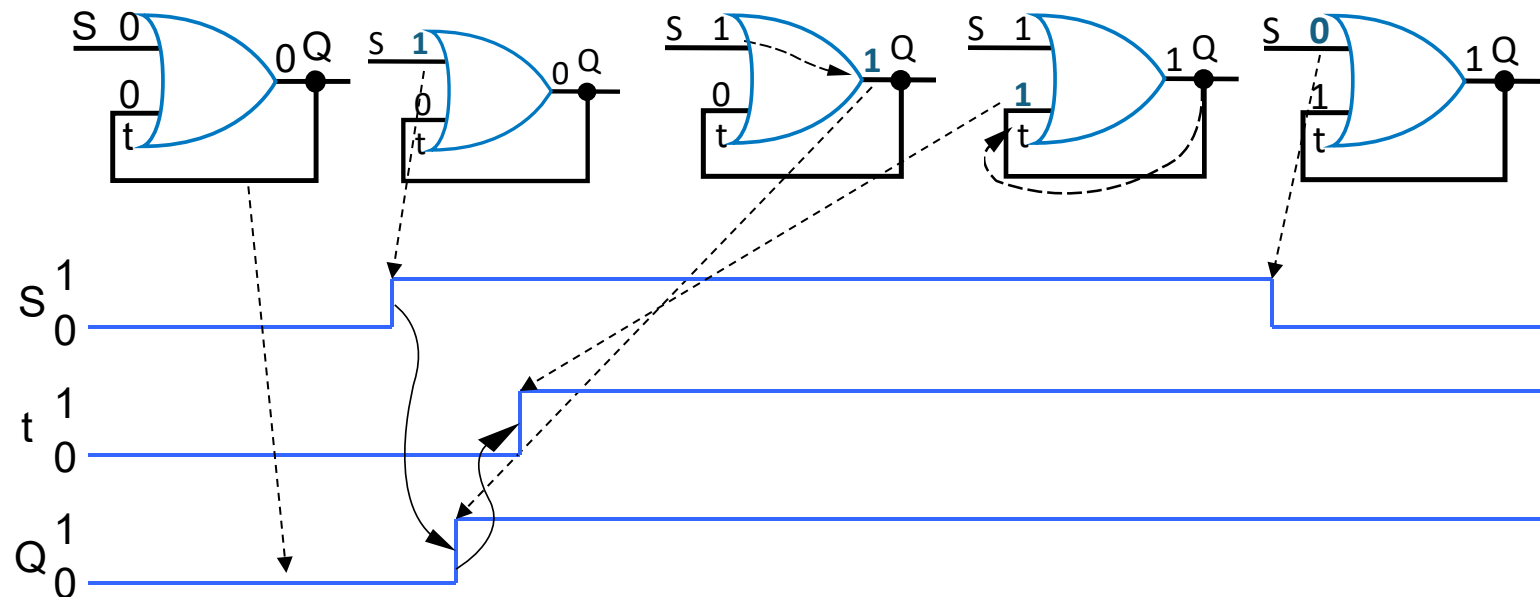
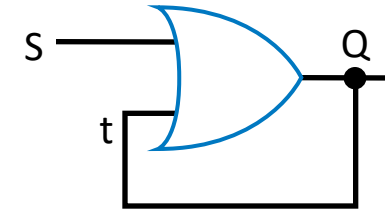


3. Cancel button pressed – light turns off



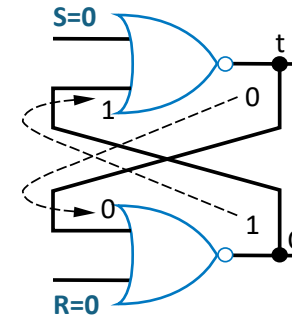
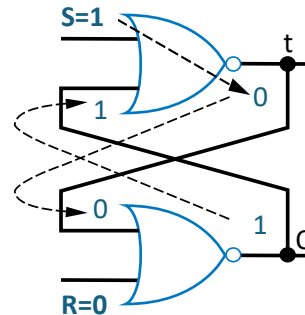
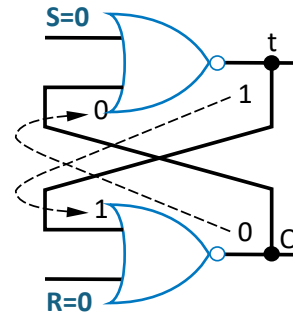
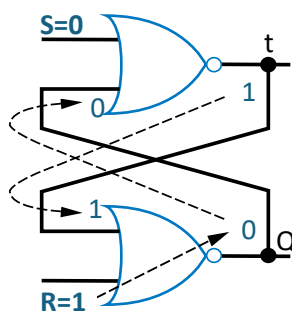
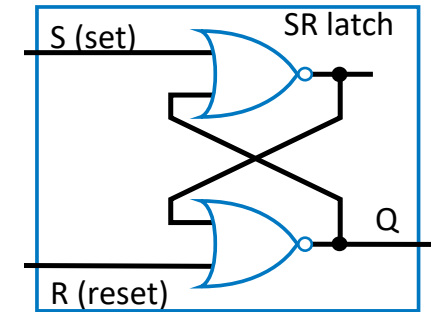
# First attempt at Bit Storage

- Need some sort of feedback
  - Does circuit on the right do what we want?
    - No: Once Q becomes 1 (when S=1), Q stays 1 forever – no value of S can bring Q back to 0

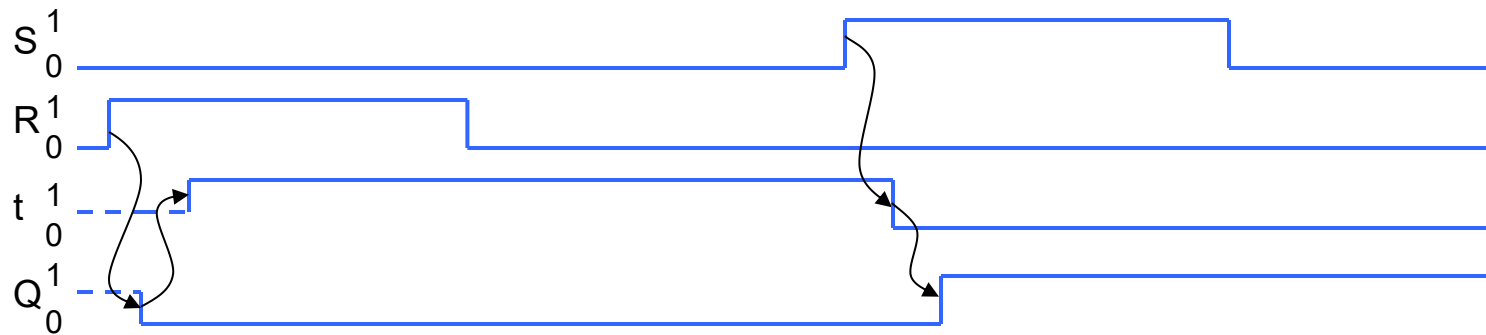
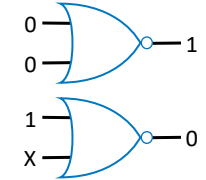


# Bit Storage Using an SR Latch

- Does the circuit to the right, with cross-coupled NOR gates, do what we want?
  - Yes! How did someone come up with that circuit? Maybe just trial and error, a bit of insight...

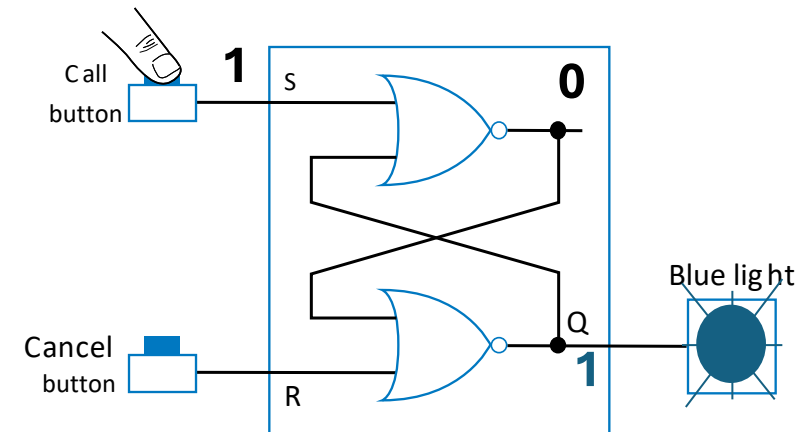
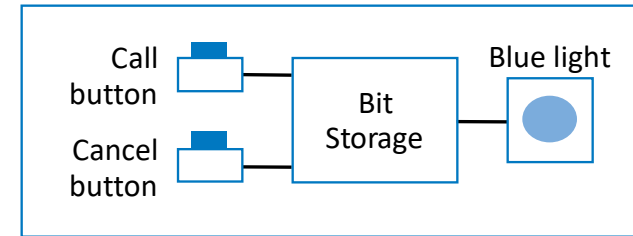


Recall NOR...



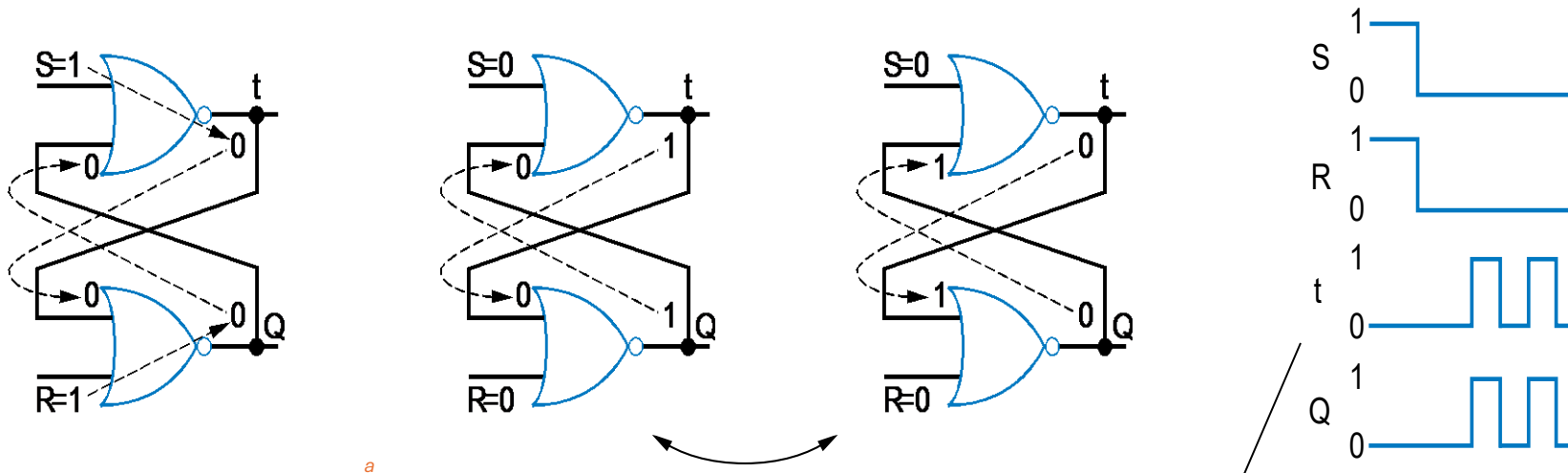
# Example Using SR Latch for Bit Storage

- SR latch can serve as bit storage in previous example of flight-attendant call button
  - Call=1 : sets Q to 1
    - Q stays 1 even after Call=0
  - Cancel=1 : resets Q to 0
- But, there's a problem...



# Problem with SR Latch

- Problem
  - If  $S=1$  and  $R=1$  simultaneously, we don't know what value  $Q$  will take

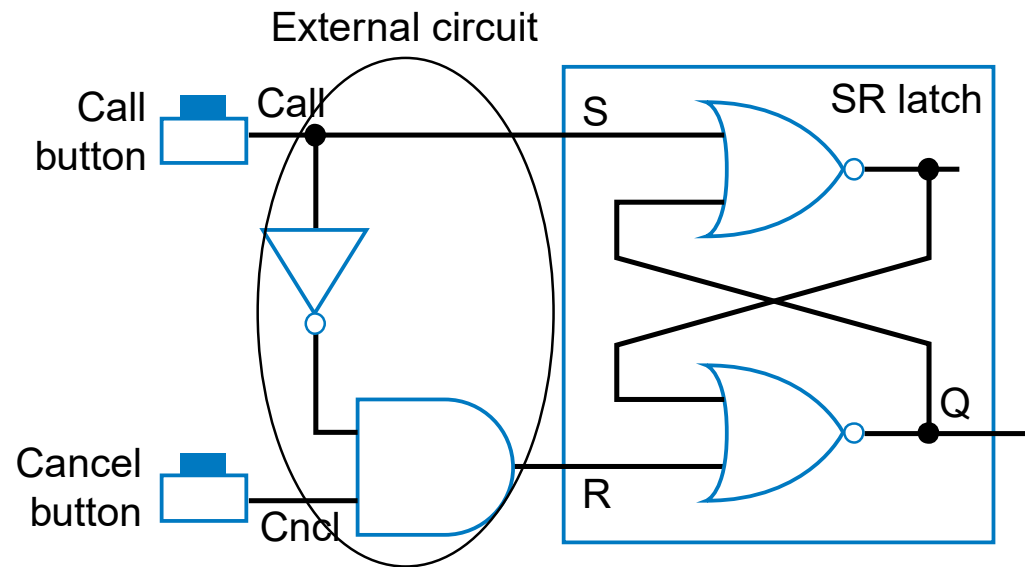


$Q$  may oscillate. Then, because one path will be slightly longer than the other,  $Q$  will eventually settle to 1 or 0 – but we don't know which. Known as a *race condition*.

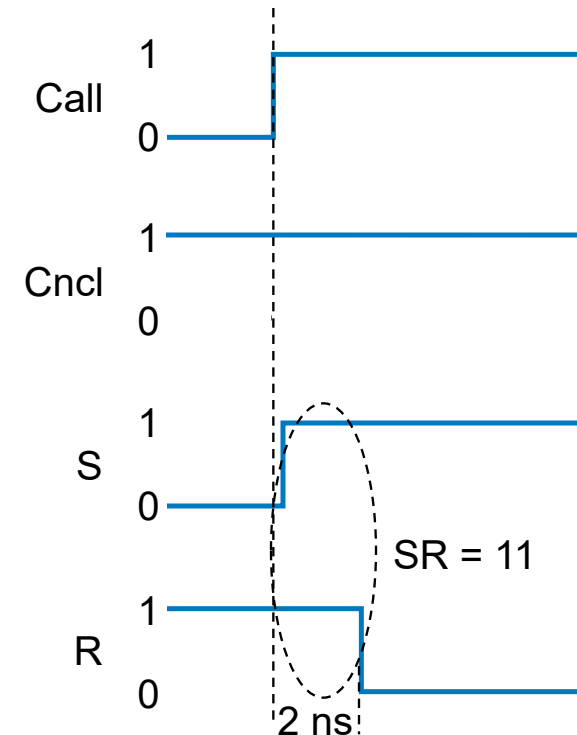


# Problem with SR Latch

- Designer might try to avoid problem using external circuit
  - Circuit should prevent SR from ever being 11
  - But 11 can occur due to different path delays

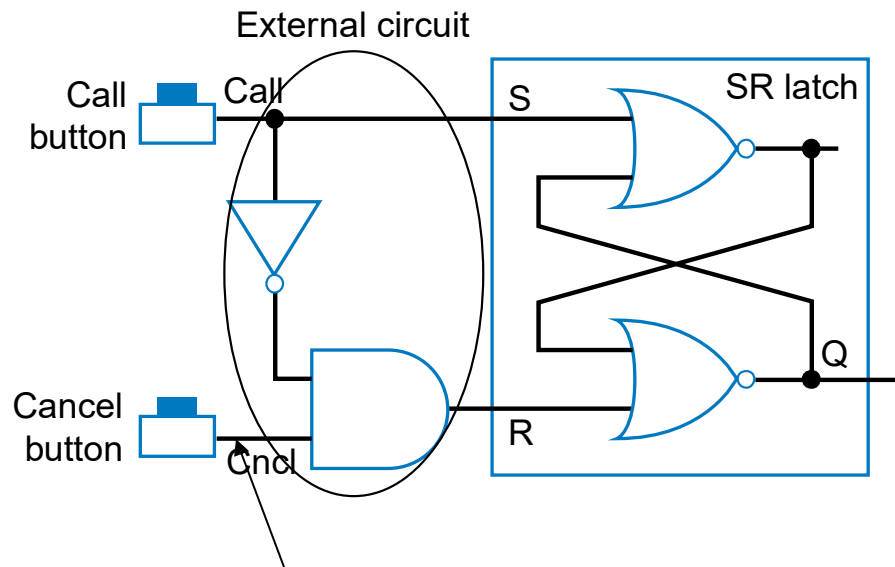


Assume 1 ns delay per gate. The longer path from Call to R than from Call to S causes  $SR=11$  for short time – could be long enough to cause oscillation

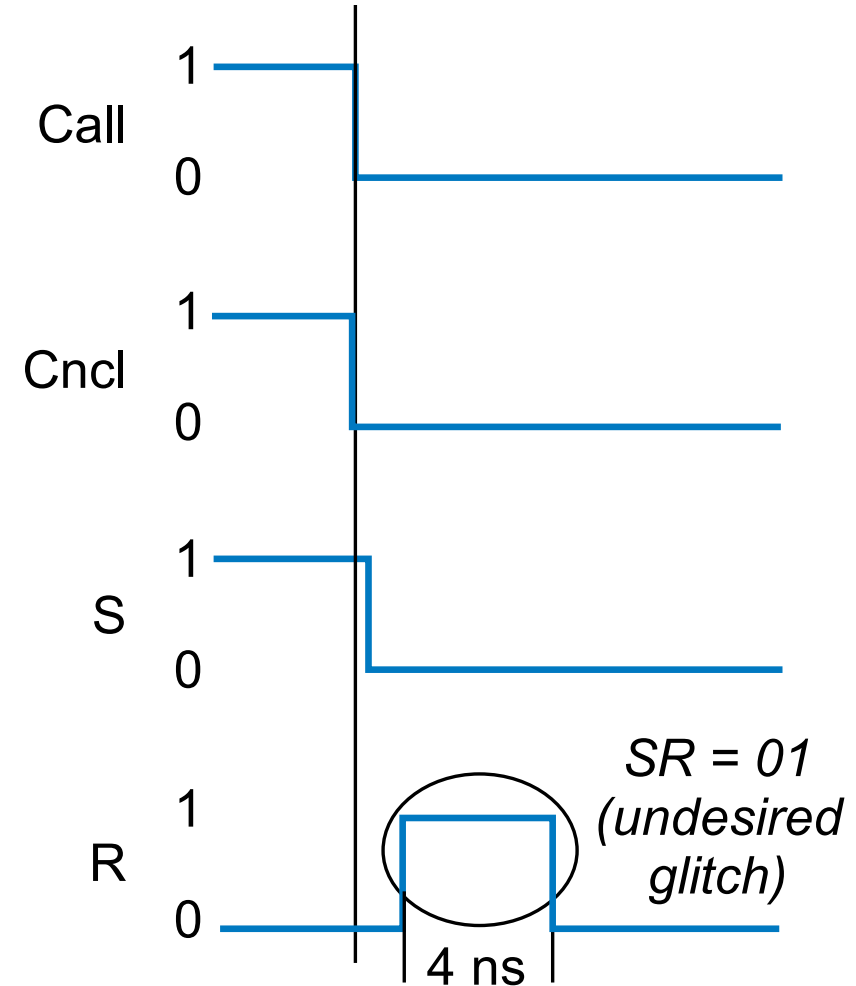


# Problem with SR Latch

- Glitch can also cause undesired set or reset

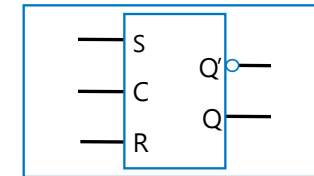
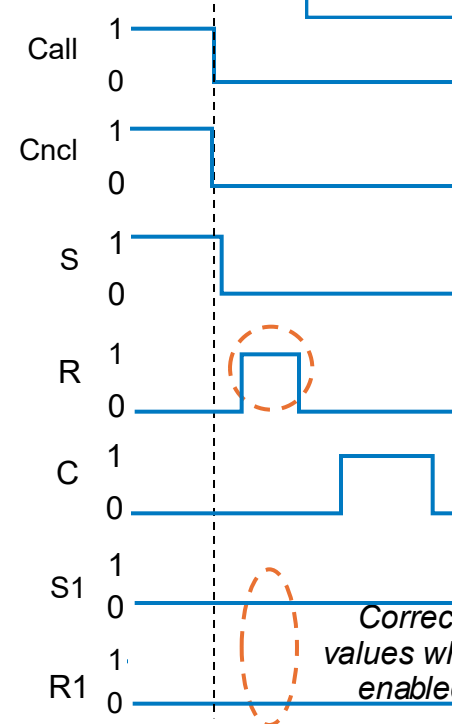
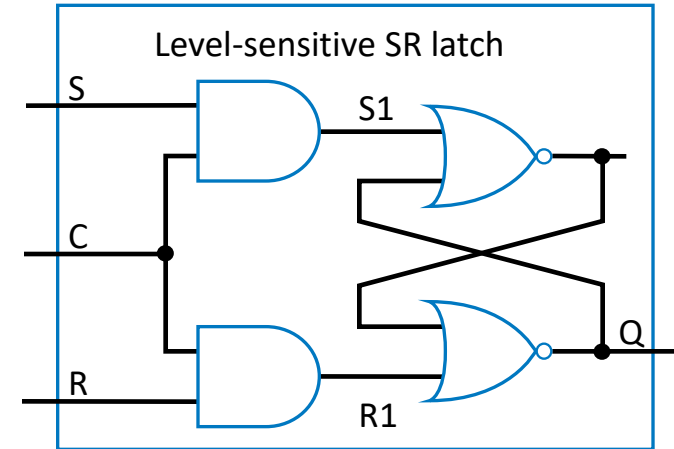
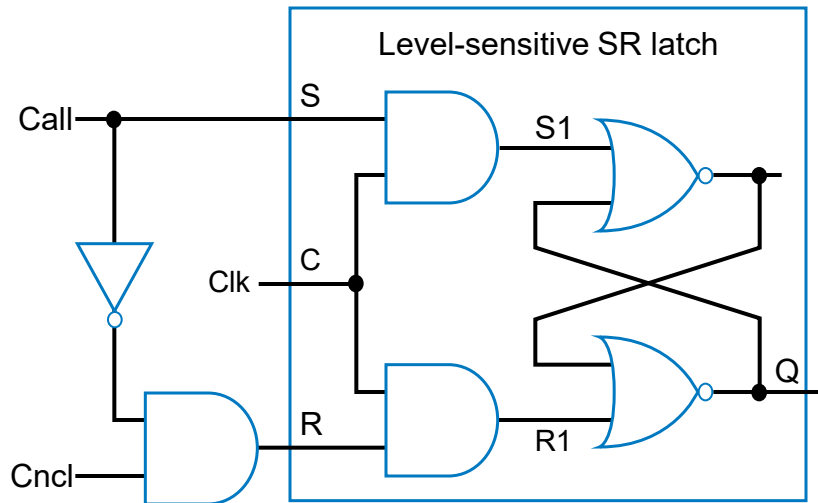


*Suppose this wire has 4 ns delay*



# Solution: Level-Sensitive SR Latch

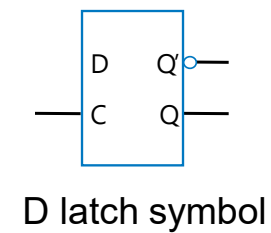
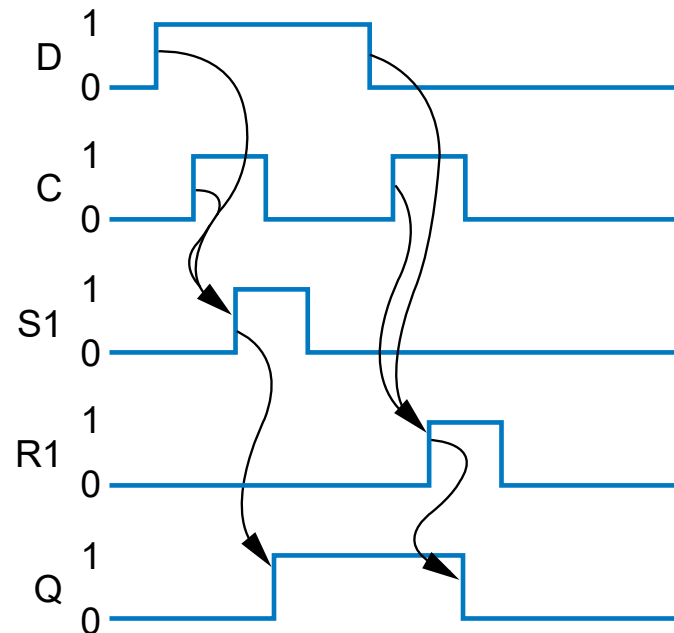
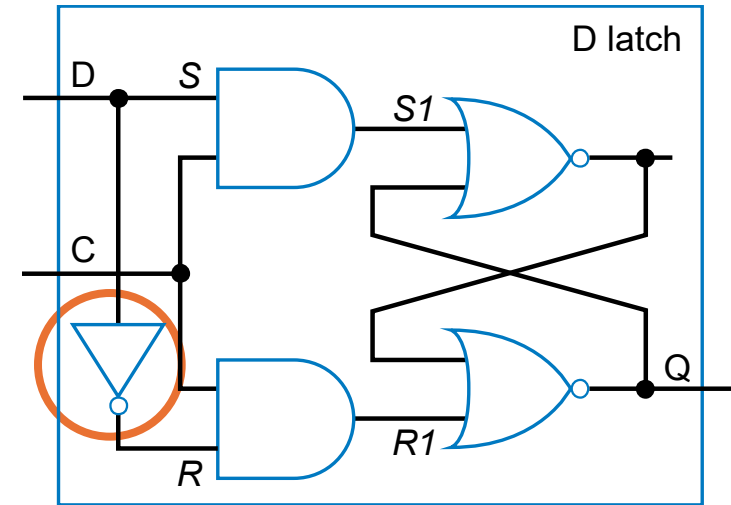
- Add enable input “C”
- Only let S and R change when C=0
  - Ensure circuit in front of SR never sets  $SR=11$ , except briefly due to path delays
  - Set C=1 after time for S and R to be stable
  - When C becomes 1, the stable S and R value passes through the two AND gates to the SR latch's S1 R1 inputs.



*Glitch on R (or S) doesn't affect R1 (or S1)*

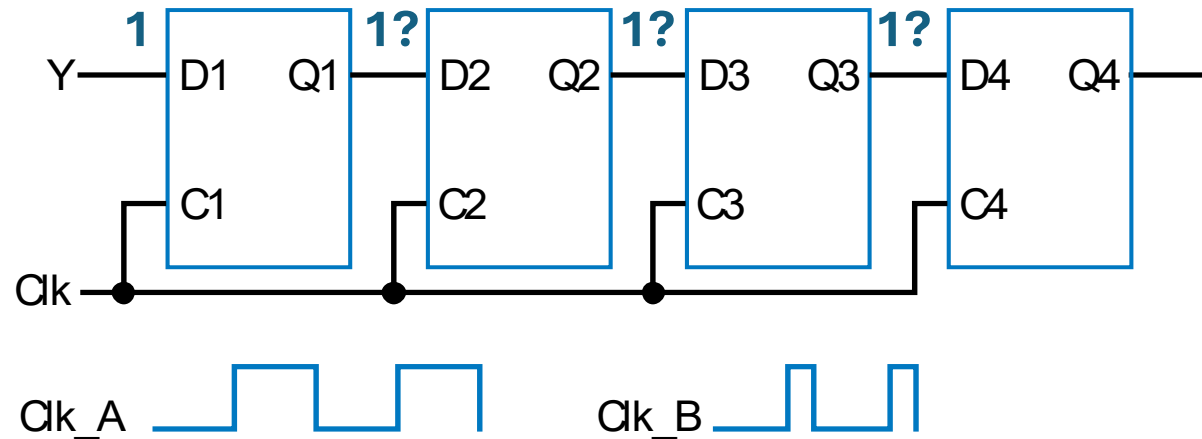
# Level-Sensitive D Latch

- SR latch requires careful design to ensure  $SR=11$  never occurs
- D latch relieves designer of that burden
  - Inserted inverter ensures R always opposite of S

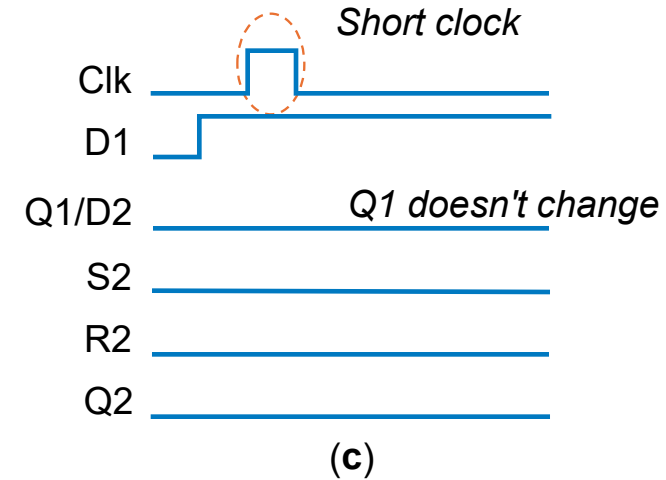
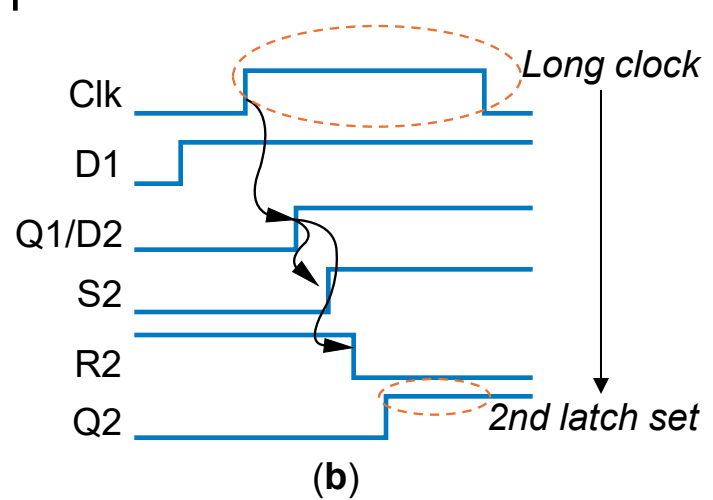
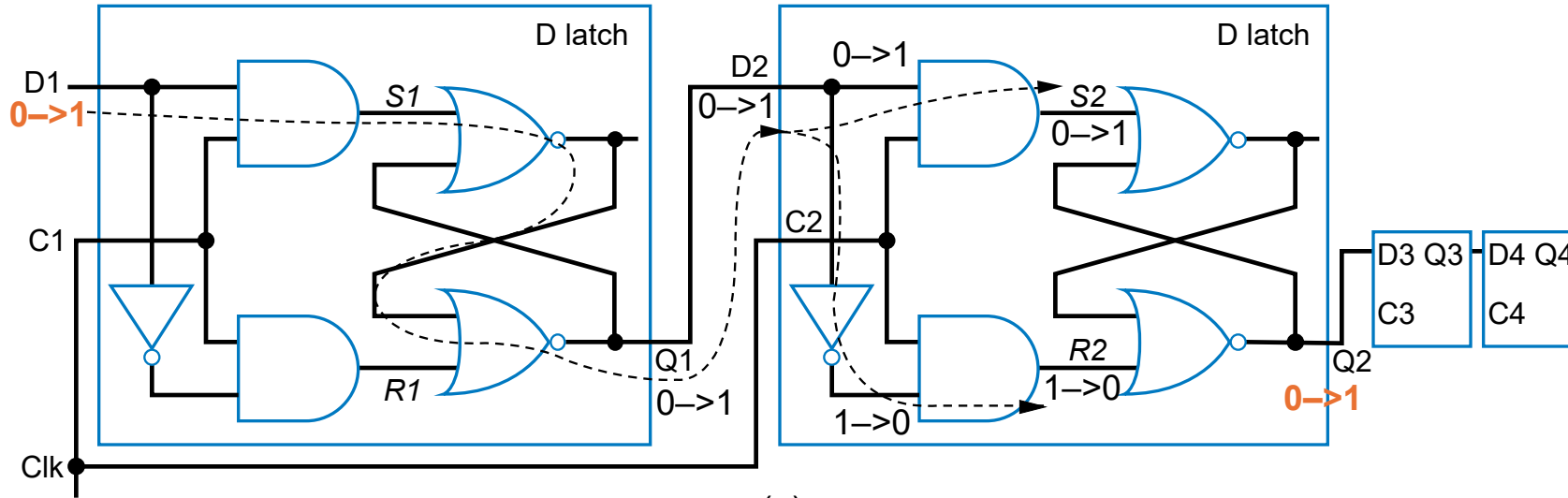


# Problem with Level-Sensitive D Latch

- D latch still has problem (as does SR latch)
  - When C=1, through how many latches will a signal travel?
  - Depends on how long C=1
    - Clk\_A – signal may travel through multiple latches
    - Clk\_B – signal may travel through fewer latches

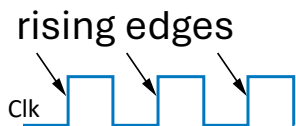


# Problem with Level-Sensitive D Latch

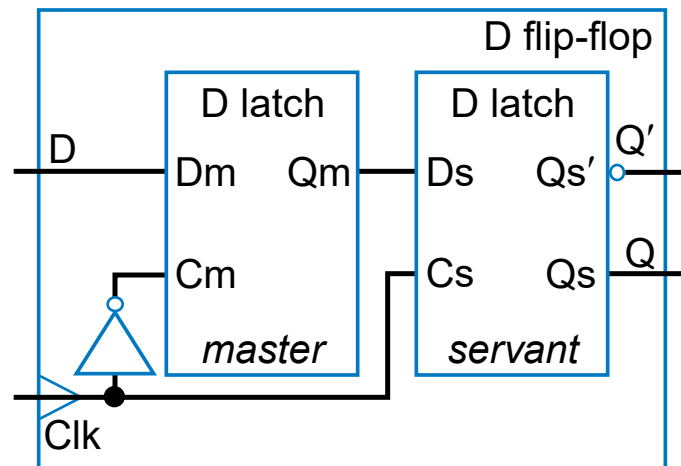


# D Flip-Flop

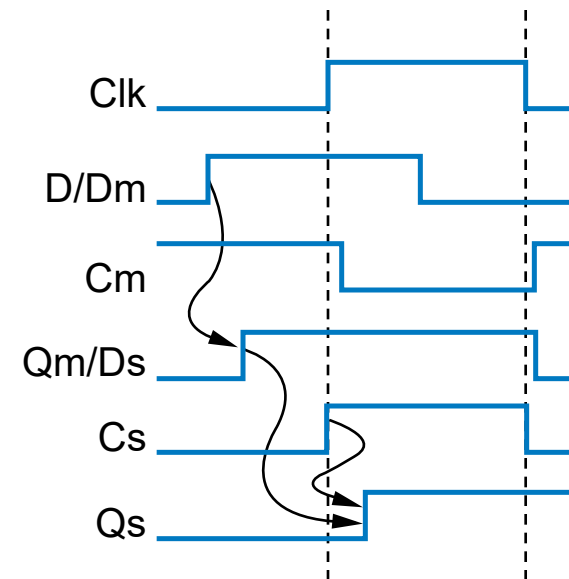
Can we design bit storage that only stores a value on the rising edge of a clock signal?



- **Flip-flop**: Bit storage that stores on clock edge
- One design – master-servant
  - Clk = 0 – master enabled, loads D, appears at Qm.
  - Clk = 1 – Master disabled, Qm stays same. Servant latch enabled, loads Qm, appears at Qs.
  - Thus, value at *D (and hence at Qm) when Clk changes from 0 to 1* gets stored into servant

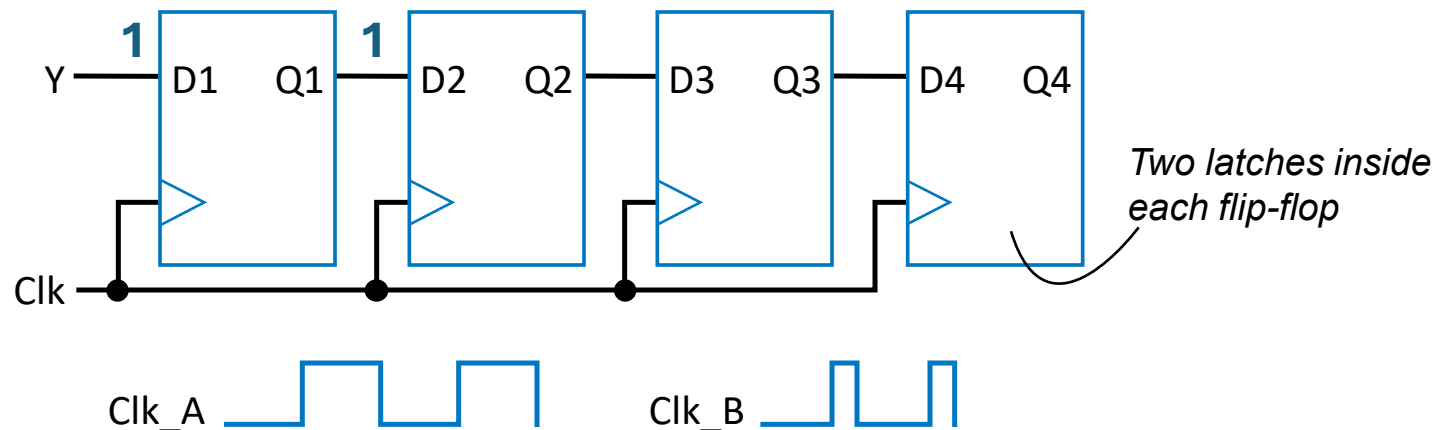


Note:  
Hundreds  
of different  
flip-flop  
designs  
exist



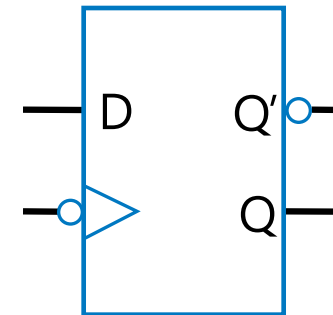
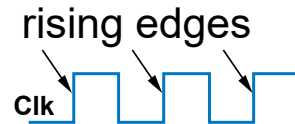
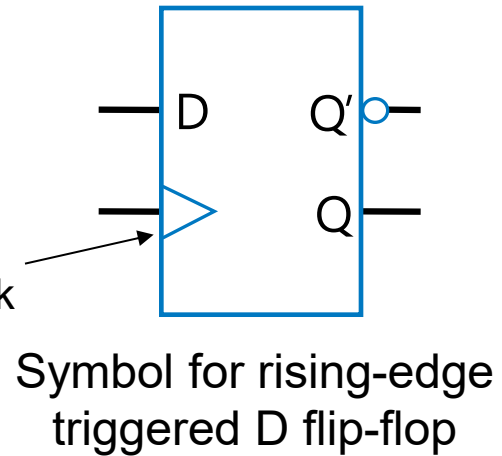
# D Flip-Flop

- Solves problem of not knowing through how many latches a signal travels when  $C=1$ 
  - In figure below, signal travels through exactly one flip-flop, for Clk\_A or Clk\_B
  - Why? Because on *rising edge* of Clk, all four flip-flops are loaded simultaneously – then all four no longer pay attention to their input, until the next rising edge. Doesn't matter how long Clk is 1.



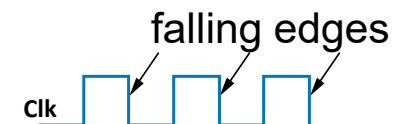
# D Flip-Flop

The triangle means edge-triggered clock input



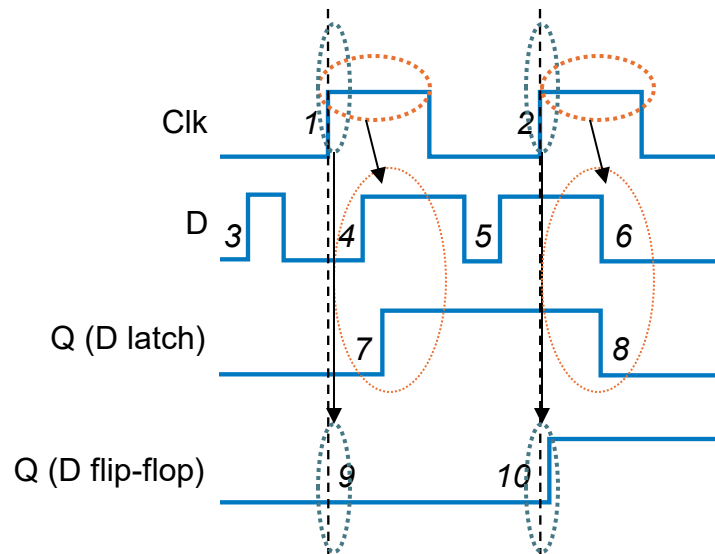
*Internal design: Just invert servant clock rather than master*

Symbol for falling-edge triggered D flip-flop



# D Latch vs. D Flip-Flop

- Latch is level-sensitive
  - Stores D when C=1
- Flip-flop is edge triggered
  - Stores D when C changes from 0 to 1
- Saying “level-sensitive latch” or “edge-triggered flip-flop” is redundant
- Comparing behavior of latch and flip-flop:



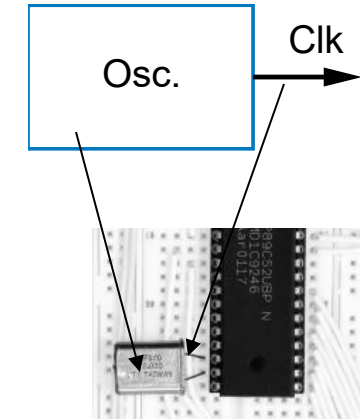
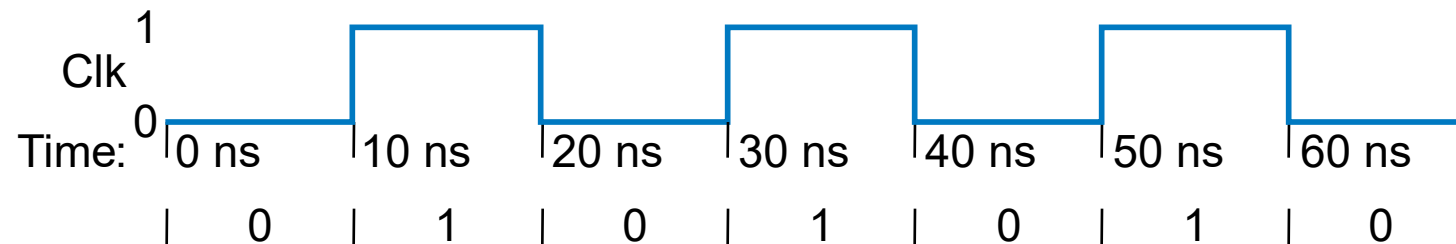
*Latch follows D  
while Clk is 1*

*Flip-flop only loads D  
during Clk rising edge*



# Clock Signal

- Flip-flop Clk inputs typically connect to one clock signal
  - Coming from an oscillator component
  - Generates periodic pulsing signal
    - Below: "Period" = 20 ns, "Frequency" =  $1/20 \text{ ns} = 50 \text{ MHz}$
    - "Cycle" is duration of 1 period (20 ns); below shows 3.5 cycles



Period/Freq shortcut: Remember 1 ns  $\rightarrow$  1 GHz

| Freq.        | Period      |
|--------------|-------------|
| 100 GHz      | 0.01 ns     |
| 10 GHz       | 0.1 ns      |
| <b>1 GHz</b> | <b>1 ns</b> |
| 100 MHz      | 10 ns       |
| 10 MHz       | 100 ns      |

# Flight-Attendant Call Button Using D Flip-Flop

- D flip-flop will store bit
- Inputs are Call, Cancel, and present value of D flip-flop, Q
- Truth table shown below

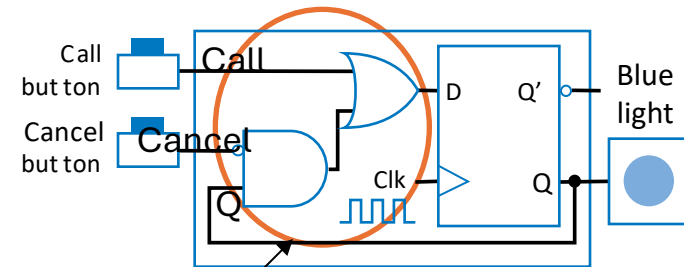
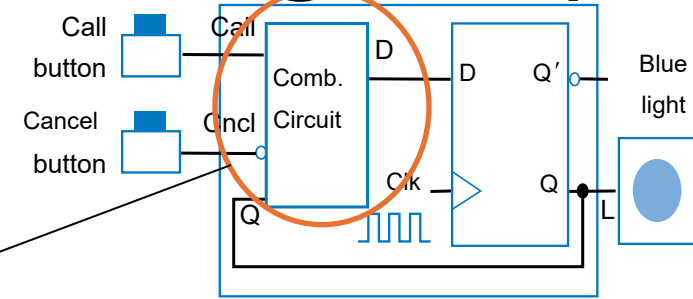
| Call | Cancel | Q | D |
|------|--------|---|---|
| 0    | 0      | 0 | 0 |
| 0    | 0      | 1 | 1 |
| 0    | 1      | 0 | 0 |
| 0    | 1      | 1 | 0 |
| 1    | 0      | 0 | 1 |
| 1    | 0      | 1 | 1 |
| 1    | 1      | 0 | 1 |
| 1    | 1      | 1 | 1 |

Preserve value: if  
Q=0, make D=0; if  
Q=1, make D=1

Cancel -- make  
D=0

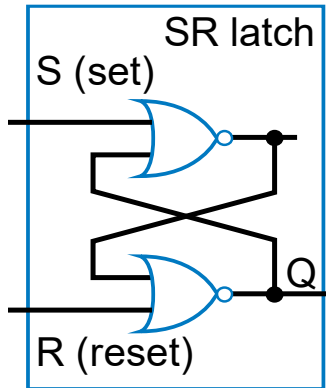
Call -- make D=1

Let's give priority  
to Call -- make  
D=1

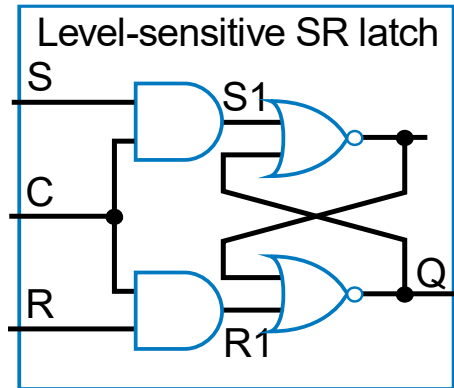


Circuit derived from truth table,  
using Chapter 2 combinational  
logic design process

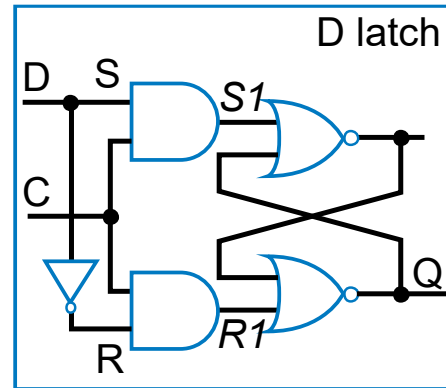
# Bit Storage Summary



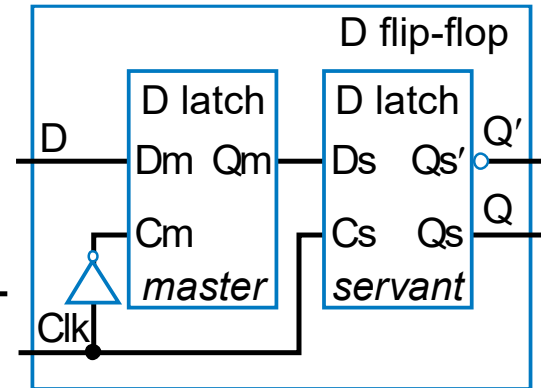
*Feature:*  $S=1$  sets  $Q$  to 1,  $R=1$  resets  $Q$  to 0.  
*Problem:*  $SR=11$  yields undefined  $Q$ , other glitches may set/reset inadvertently.



*Feature:*  $S$  and  $R$  only have effect when  $C=1$ . An external circuit can prevent  $SR=11$  when  $C=1$ .  
*Problem:* avoiding  $SR=11$  can be a burden.



*Feature:*  $SR$  can't be 11.  
*Problem:*  $C=1$  for too long will propagate new values through too many latches; for too short may not result in the bit being stored.

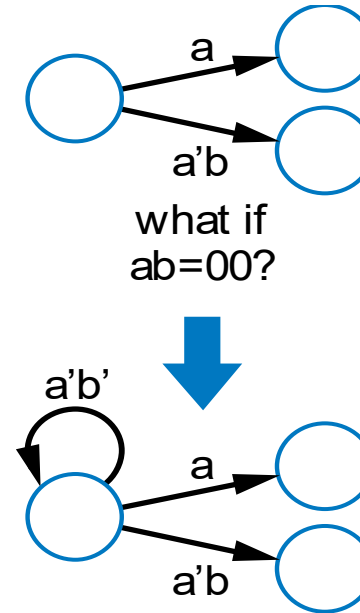
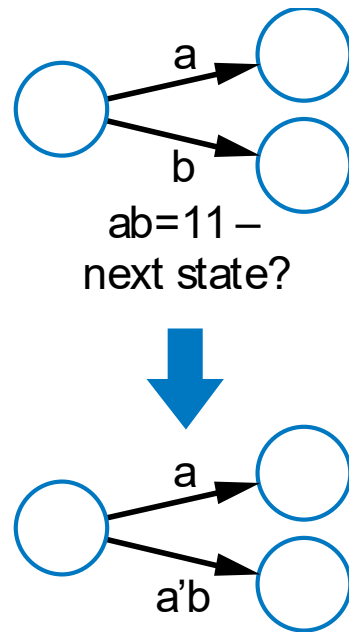


*Feature:* Only loads  $D$  value present at rising clock edge, so values can't propagate to other flip-flops during same clock cycle. *Tradeoff:* uses more gates internally, and requires more external gates than  $SR$ —but transistors today are more plentiful and cheaper.

- We considered increasingly better bit storage until we arrived at the robust D flip-flop bit storage

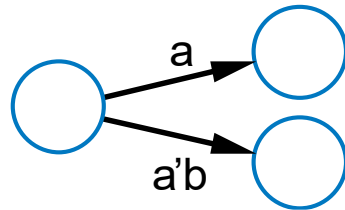
# Common Mistakes when Capturing FSMs

- Non-exclusive transitions
- Incomplete transitions



# Verifying Correct Transition Properties

- Can verify using Boolean algebra
  - Only one condition true: AND of each condition pair (for transitions leaving a state) should equal 0  $\rightarrow$  proves pair can never simultaneously be true
  - One condition true: OR of all conditions of transitions leaving a state) should equal 1  $\rightarrow$  proves at least one condition must be true
  - Example



Answer:

$$\begin{aligned} & a * a'b \\ &= (a * a') * b \\ &= 0 * b \\ &= 0 \\ &\text{OK!} \end{aligned}$$

$$\begin{aligned} & a + a'b \\ &= a*(1+b) + a'b \\ &= a + ab + a'b \\ &= a + (a+a')b \\ &= a + b \end{aligned}$$

Fails! Might not be 1 (i.e.,  $a=0$ ,  $b=0$ )

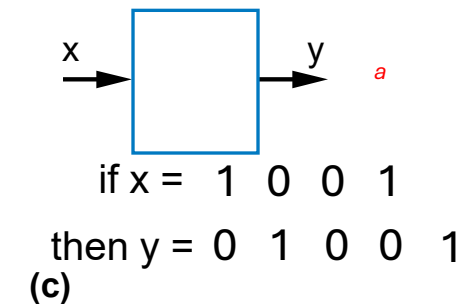
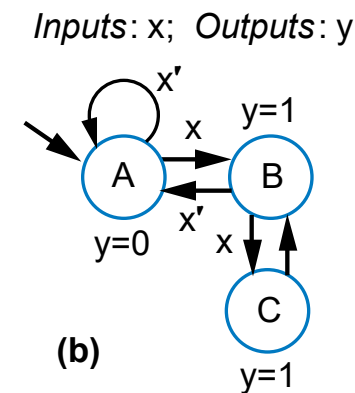
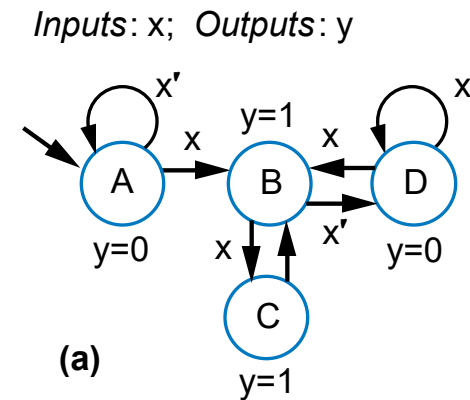
Q: For shown transitions, prove whether:

- \* Only one condition true (AND of each pair is always 0)
- \* One condition true (OR of all transitions is always 1)



# Sequential Logic Optimization and Tradeoffs

- **State Reduction:**  
Reduce number of states in FSM *without* changing behavior
  - Fewer states potentially reduces size of state register and combinational logic

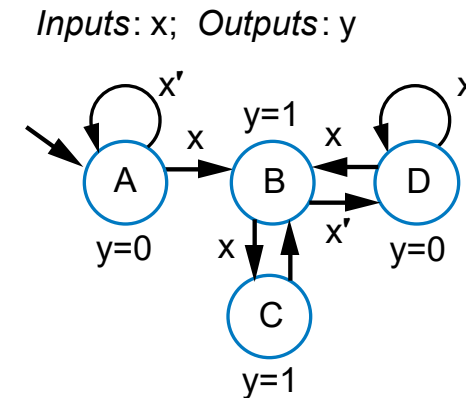


*For the same sequence of inputs, the output of the two FSMs is the same*

# State Reduction: Equivalent States

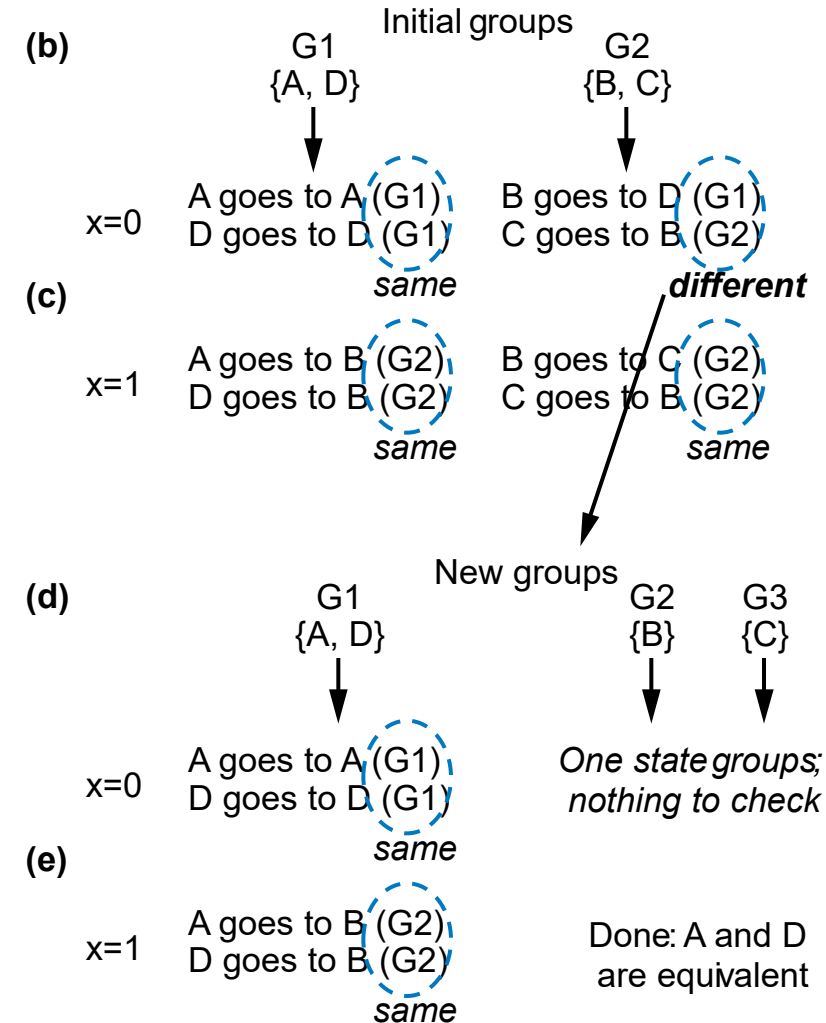
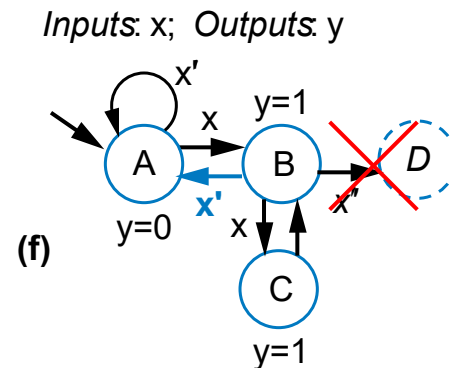
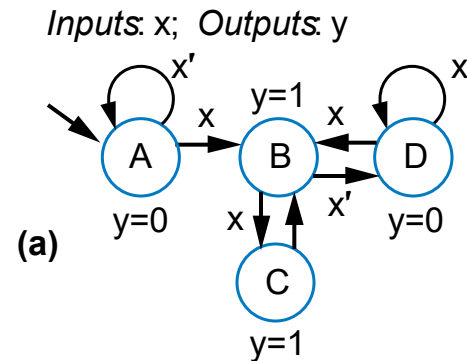
Two states are equivalent if:

1. They assign the same values to outputs
  - e.g. **A** and **D** both assign  $y$  to 0,
2. AND, for *all* possible sequences of inputs, the FSM outputs will be the same starting from either state
  - e.g. say  $x=1,1,0$ 
    - starting from **A**,  $y=0,1,1,1,\dots$  (A,B,C,B)
    - starting from **D**,  $y=0,1,1,1,\dots$  (D,B,C,B)

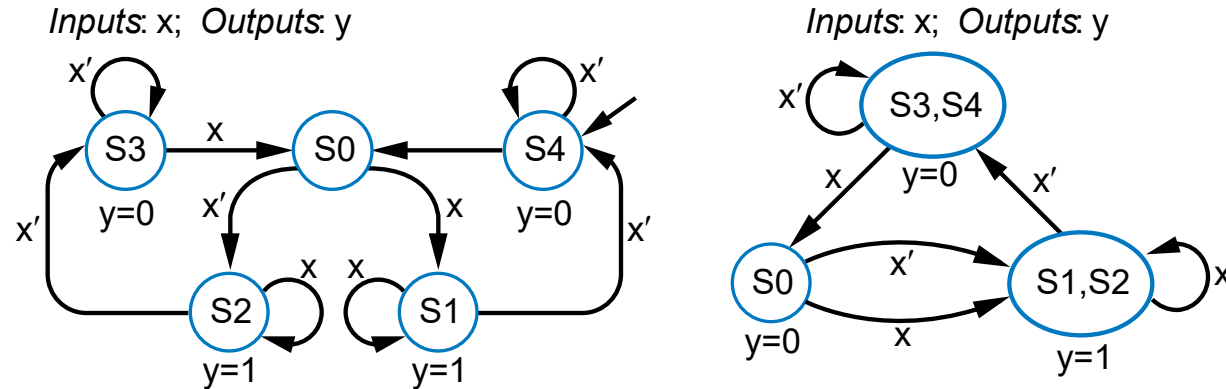


# State Reduction via the Partitioning Method

- First partition states into groups based on outputs
- Then partition based on next states
  - For each possible input, for states in group, if next states in different groups, divide group
  - Repeat until no such states



# Ex: Minimizing States using the Partitioning Method

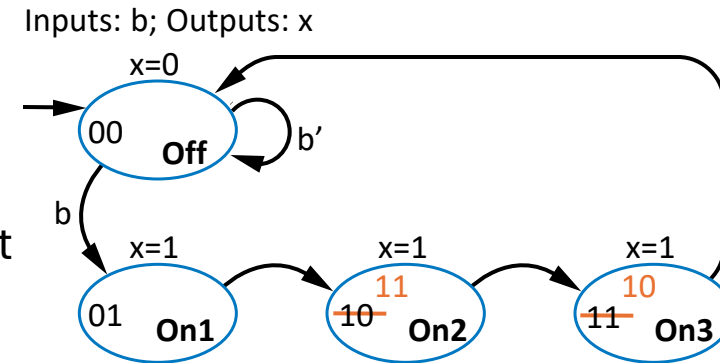


- Initial grouping based on outputs:  $G1=\{S3, S0, S4\}$  and  $G2=\{S2, S1\}$
- Group  $G1$ : Differ  $\rightarrow$  divide
  - for  $x=0$ ,  $S3 \rightarrow S3$  ( $G1$ ),  $S0 \rightarrow S2$  ( $G2$ ),  $S4 \rightarrow S0$  ( $G1$ )
  - for  $x=1$ ,  $S3 \rightarrow S0$  ( $G1$ ),  $S0 \rightarrow S1$  ( $G2$ ),  $S4 \rightarrow S0$  ( $G1$ )
  - Divide  $G1$ . New groups:  $G1=\{S3,S4\}$ ,  $G2=\{S2,S1\}$ , and  $G3=\{S0\}$
- Repeat for  $G1$ , then  $G2$ . ( $G3$  only has one state). No further divisions, so done.
  - $S3$  and  $S4$  are equivalent
  - $S2$  and  $S1$  are equivalent



# State Encoding

- **State encoding**: Assigning unique bit representation to each state
- Different encodings may reduce circuit size, or trade off size and performance
- **Minimum-bitwidth binary encoding**: Uses fewest bits
  - Alternatives still exist, such as
    - A:00, B:01, C:10, D:11
    - A:00, B:01, C:11, D:10
  - $4! = 24$  alternatives for 4 states,  $N!$  for  $N$  states
- Consider Three-Cycle Laser Timer...
  - Example 3.7's encoding led to **15** gate inputs
  - Try **alternative encoding**
    - $x = s1 + s0$
    - $n1 = s0$
    - $n0 = s1'b + s1's0$
    - Only **8** gate inputs
    - Thus fewer transistors



|     | Inputs |                |   | Outputs |    |                |
|-----|--------|----------------|---|---------|----|----------------|
|     | s1     | s0             | b | x       | n1 | n0             |
| Off | 0      | 0              | 0 | 0       | 0  | 0              |
|     | 0      | 0              | 1 | 0       | 0  | 1              |
| On1 | 0      | 1              | 0 | 1       | 1  | <del>0</del> 1 |
|     | 0      | 1              | 1 | 1       | 1  | <del>0</del> 1 |
| On2 | 1      | <del>0</del> 1 | 0 | 1       | 1  | <del>1</del> 0 |
|     | 1      | <del>0</del> 1 | 1 | 1       | 1  | <del>1</del> 0 |
| On3 | 1      | <del>1</del> 0 | 0 | 1       | 0  | 0              |
|     | 1      | <del>1</del> 0 | 1 | 1       | 0  | 0              |



# State Encoding: One-Hot Encoding

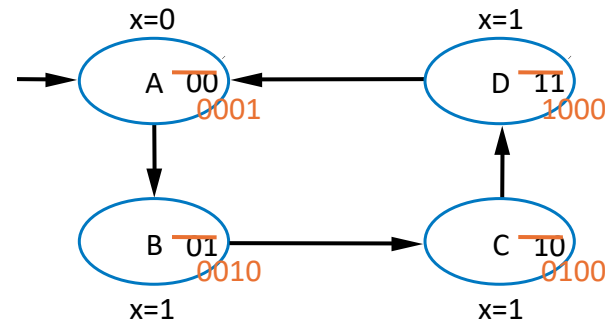
- **One-hot encoding**

- One bit per state – a bit being '1' corresponds to a particular state
- For A, B, C, D:  
A: 0001, B: 0010, C: 0100, D: 1000

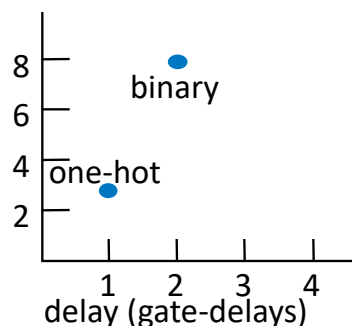
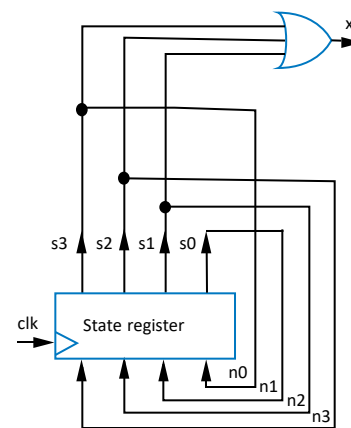
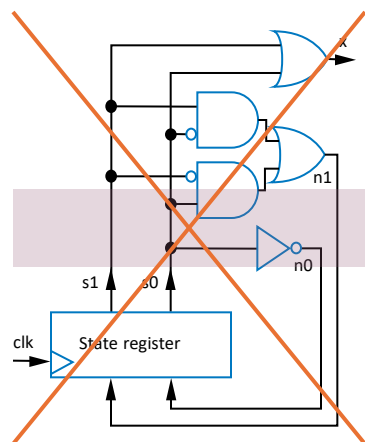
- Example: FSM that outputs 0, 1, 1, 1

- Equations if one-hot encoding:
  - $n3 = s2$ ;  $n2 = s1$ ;  $n1 = s0$ ;  $x = s3 + s2 + s1$
- Fewer gates and only one level of logic – less delay than two levels, so faster clock frequency

Inputs: none; Outputs: x


~~|   | Inputs |    | Outputs |    |   |
|---|--------|----|---------|----|---|
|   | s1     | s0 | n1      | n0 | x |
| A | 0      | 0  | 0       | 1  | 0 |
| B | 0      | 1  | 1       | 0  | 1 |
| C | 1      | 0  | 1       | 1  | 1 |
| D | 1      | 1  | 0       | 0  | 1 |~~

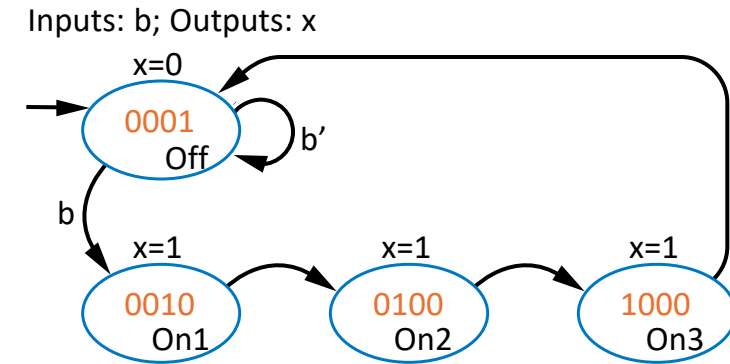
|   | Inputs |    |    |    | Outputs |    |    |    |   |
|---|--------|----|----|----|---------|----|----|----|---|
|   | s3     | s2 | s1 | s0 | n3      | n2 | n1 | n0 | x |
| A | 0      | 0  | 0  | 1  | 0       | 0  | 1  | 0  | 0 |
| B | 0      | 0  | 1  | 0  | 0       | 1  | 0  | 0  | 1 |
| C | 0      | 1  | 0  | 0  | 1       | 0  | 0  | 0  | 1 |
| D | 1      | 0  | 0  | 0  | 0       | 0  | 0  | 1  | 1 |



# One-Hot Encoding Example:

## Three-Cycles-High Laser Timer

- Four states – Use four-bit one-hot encoding
  - State table leads to equations:
    - $x = s_3 + s_2 + s_1$
    - $n_3 = s_2$
    - $n_2 = s_1$
    - $n_1 = s_0 \cdot b$
    - $n_0 = s_0 \cdot b' + s_3$
  - Smaller
    - $3+0+0+2+(2+2) = \mathbf{9}$  gate inputs
    - Earlier binary encoding (Ch 3): **15** gate inputs
  - Faster
    - Critical path:  $n_0 = s_0 \cdot b' + s_3$
    - Previously:  $n_0 = s_1' s_0' b + s_1 s_0'$
    - 2-input AND slightly faster than 3-input AND

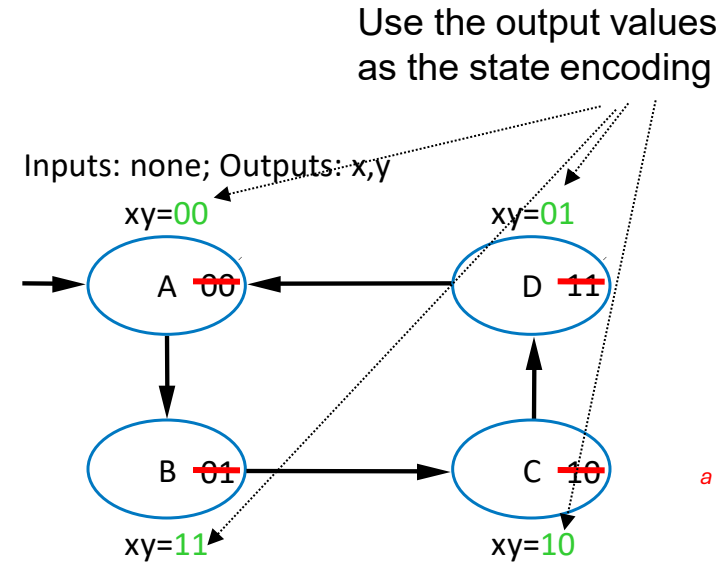


|     | Inputs |    |    |    |   | Outputs |    |    |    |    |
|-----|--------|----|----|----|---|---------|----|----|----|----|
|     | s3     | s2 | s1 | s0 | b | x       | n3 | n2 | n1 | n0 |
| Off | 0      | 0  | 0  | 1  | 0 | 0       | 0  | 0  | 0  | 1  |
|     | 0      | 0  | 0  | 1  | 1 | 0       | 0  | 0  | 1  | 0  |
| On1 | 0      | 0  | 1  | 0  | 0 | 1       | 0  | 1  | 0  | 0  |
|     | 0      | 0  | 1  | 0  | 1 | 1       | 0  | 1  | 0  | 0  |
| On2 | 0      | 1  | 0  | 0  | 0 | 1       | 1  | 0  | 0  | 0  |
|     | 0      | 1  | 0  | 0  | 1 | 1       | 1  | 0  | 0  | 0  |
| On3 | 1      | 0  | 0  | 0  | 0 | 1       | 0  | 0  | 0  | 1  |
|     | 1      | 0  | 0  | 0  | 1 | 1       | 0  | 0  | 0  | 1  |



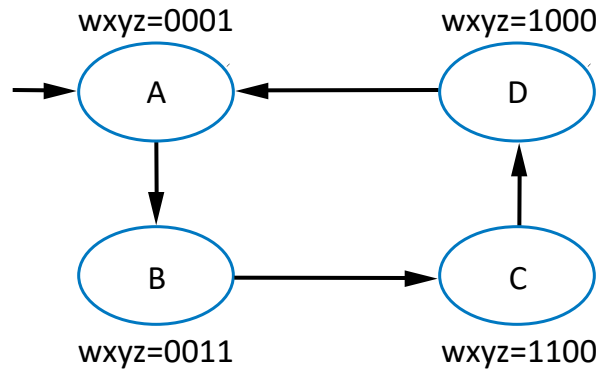
# Output Encoding

- **Output encoding:**  
Encoding method where the state encoding is same as the **output values**
  - Possible if enough outputs, all states with unique output values



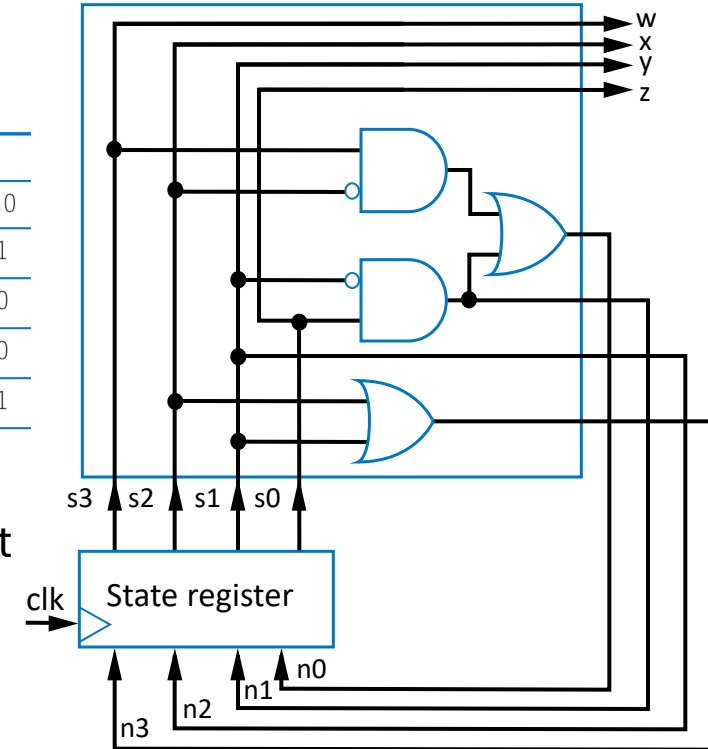
# Output Encoding Example: Sequence Generator

Inputs: none; Outputs: w, x, y, z

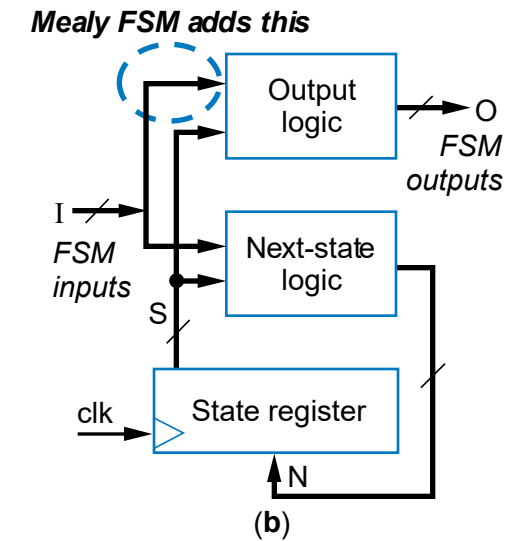
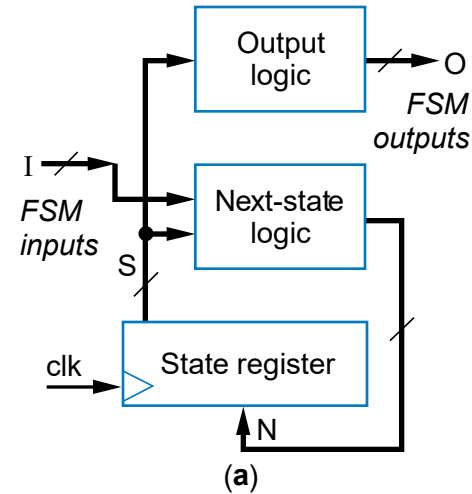
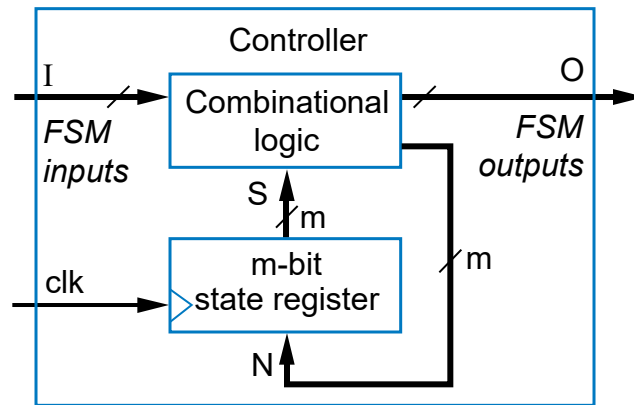


|   | Inputs |    |    |    | Outputs |    |    |    |
|---|--------|----|----|----|---------|----|----|----|
|   | s3     | s2 | s1 | s0 | n3      | n2 | n1 | n0 |
| A | 0      | 0  | 0  | 1  | 0       | 0  | 1  | 1  |
| B | 0      | 0  | 1  | 1  | 1       | 1  | 0  | 0  |
| C | 1      | 1  | 0  | 0  | 1       | 0  | 0  | 0  |
| D | 1      | 0  | 0  | 0  | 0       | 0  | 0  | 1  |

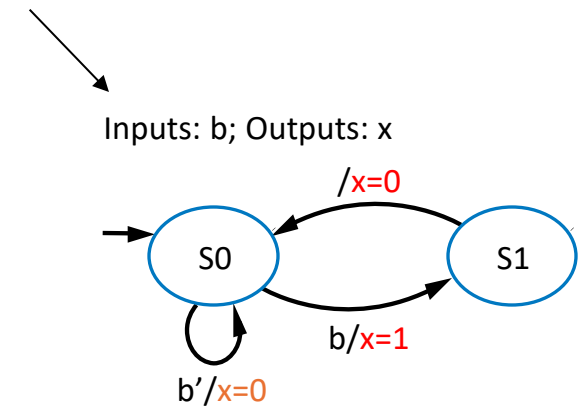
- Generate sequence 0001, 0011, 1110, 1000, repeat
  - FSM shown
- Use output values as state encoding
- Create state table
- Derive equations for next state
  - $n3 = s1 + s2$ ;  $n2 = s1$ ;  $n1 = s1's0$ ;  
 $n0 = s1's0 + s3s2'$



# Moore vs. Mealy FSMs



- FSM implementation architecture
  - State register and logic
  - More detailed view
    - Next state logic – function of present state and FSM inputs
    - Output logic
      - If function of present state only – **Moore FSM**
      - If function of present state and FSM inputs – **Mealy FSM**



Graphically: show **outputs** with transitions, not with states